new fortune cookie:

```
Item {
    id: mainForm;
    anchors.fill: parent;

    GridLayout {
        rows: 2;
        columns: 1;
        rowSpacing: 32;

        anchors.centerIn: parent;

        Label {
            id: mainFortuneLabel;
            Layout.maximumWidth: 600;
            Layout.alignment: Qt.AlignHCenter |
Qt.AlignVCenter;

            text: qsTr("Set a server, and click the Get
Fortune button");
            wrapMode: Text.Wrap;
            horizontalAlignment: Text.AlignHCenter;
            font.pointSize: 36;

            function setText(mtext) {
                text = qsTr(mtext);
                horizontalAlignment = Text.AlignLeft;
                horizontalAlignment = Text.AlignHCenter;
            }
        }

        Button {
            id: getFortuneButton;
            Layout.alignment: Qt.AlignHCenter |
Qt.AlignVCenter;

            text: "Get Fortune";
            onClicked: fortuneClient.getNewFortune();
        }
    }
```

This implements the main form. Notice that we have a *setText()* function as a part of the main label to change its text. This will be triggered by the *Fortune* component when it obtains a new fortune cookie from the server.

We've also had to reset the horizontal alignment every time we change the text. This is because every time the text is changed, the new text is rendered with a very weird alignment. Whether this is a bug or by design is unclear, but resetting the alignment seems to fix this.

The next few lines implement the server selection dialogue box. Again, this is fairly simple but somewhat long, and to save space, we won't reproduce it in print.

At this point, you might find QML overwhelming—let's face it, we've already dealt with about 10 components and a lot more properties of each. The truth is, QML's component sets are pretty huge, and to study each one of them in-depth before writing code is impractical. Hence this—an application in QML to get you started, and the component reference manuals to refer to as you go along, which are linked to at the end of the article.

This brings us to the one component that we must build ourselves, in order to make this GUI talk to the fortune server. This is what it looks like:

```
FortuneClient {
    id: fortuneClient;
    onHaveFortune: mainFortuneLabel.setText(fortune);

    function setServerPort() {
        serverHost = serverTextField.text;
        serverPort = portTextField.text;

        statusBarText.text = qsTr("OK: Server set to %1:%2".
arg(serverHost).arg(serverPort));
    }
}
```

And to explain how it works will involve another theory class and a bit of C++ coding.

The QQuickItem and the QQmlApplicationEngine

To load a QML-based UI into a Qt application, you start by creating a master *QApplication* instance (like all other Qt applications), but then you create a *QQmlApplicationEngine* instance, into which you load up your QML file. This engine executes your QML script and takes care of all the plumbing for you.

In actual code, it looks something like this:

```
QApplication app;
QQmlApplicationEngine engine;
engine.load(QUrl(QStringLiteral("qrc:/main.qml")));
```

True, it's just three lines. The *engine* provides a standard QtQuick environment, so all QML components that are installed on the user's system are available for the programmer to use.

But what about custom components? We'll need to use one, so let's figure out how to inject one into the *QQmlApplicationEngine*. It turns out that like the QApplication instance (which works at the global level and doesn't need to be touched), the *QQmlApplicationEngine* is also a global object, into which we can inject custom components at will.

Now what about those components themselves? Well, technically, any object that inherits from *QObject* and implements the meta-object system can be used as a QML component. Remember, QML components don't have to be visible GUI widgets